Homogeneous variadic function parameters

Document number:	P1219R2
Date:	2019-10-05
Project:	Programming Language C++, Evolution Working Group
Feature-test macros:	`__cpp_variadic_templates`

Reply-to: James Touton <bekenn@gmail.com>

Table of Contents

Revision History

Revision 2 - 2019-10-05

- Rebased wording against N4830.
- Added discussion on the impact of making varargs commas mandatory.

Revision 1 - 2019-03-11

- Rebased wording against N4800.
- Added wording for signature changes.
- Removed wording permitting empty template parameter lists for template declarations.
- Added feature test macro.
- Added discussion of alternative designs aimed at addressing or avoiding confusion with explicit specializations.
- Added straw polls from EWGI in Kona.
- Added acknowledgments.
- Minor corrections to examples.

Revision 0 - 2018-10-08

- Initial draft.

Introduction

This paper seeks to expand the usefulness of variadic templates by allowing variadic function parameter packs to be declared using a single type. This would make the following construct legal:

```
template <class T>  // T is not a parameter pack...
void foo(T... vs);  // ...but vs is a parameter pack.
```

At first glance, this seems a simple and natural extension to variadic templates; after all, the following is already legal:

```
// Legal since C++11: T is not a parameter pack, but vs is.
template <class T, T... vs>
void foo();
```

In both examples, `vs` is a parameter pack, with each element having the type `T`. The meaning follows naturally from the rules for variadic templates, so it is a bit surprising that one is legal and the other is not. This paper explores the design space around making the first example legal.

Straw Polls

A draft revision of this paper was seen by EWGI in Kona in March of 2019.

EWGI Poll: Make vararg declaration comma mandatory.

SF	F	N	A	SA
10	6	1	0	0

EWGI Poll: This should be valid and declare a template: `template <class T> constexpr T min(T v1, T... vs);`

SF	F	N	A	SA

10	5	1	0	0

EWGI Poll: This should be valid and declare a template: `template <> void foo(int... v);`

SF	F	N	A	SA
0	4	7	2	3

EWGI Poll: This should be valid and declare a template: `void foo(int... v);`

SF	F	N	A	SA
4	6	4	0	1

A follow-up revision incorporating EWGI's feedback was seen by EWGI in Cologne in July of 2019.

EWGI Poll: Forward to EWG.

SF	F	N	A	SA
5	2	3	0	0

Definitions

Throughout this paper, the term *homogeneous parameter pack* (or *homogeneous pack*) will be used to refer to a parameter pack containing values, all of the same type, where the pack's declaration does not expand a pack of types. A parameter pack containing values that are permitted to be of different types is referred to as a *heterogeneous parameter pack* (or *heterogeneous pack*). The term *homogeneous function parameter pack* refers to a homogeneous pack of function parameters, and the term *homogeneous template parameter pack* refers to a homogeneous pack of template parameters. The terms *heterogeneous function parameter pack* and *heterogeneous template parameter pack* refer to heterogeneous packs of function and template parameters, respectively.

Implementation Experience

An implementation of this proposal based on Clang can be found at https://github.com/Bekenn/clang/tree/func-parm-packs. It is believed to be very nearly both complete and correct, with only minor deviations in behavior from the wording provided in this paper.

Motivation and Scope

Meeting user expectations

This table showcases the current lack of symmetry between template parameter packs and function parameter packs:

	Template parameter pack	Function parameter pack

Heterogeneous	`// OK:`
	`template <auto... vs>`
	`void f();`
		`// OK:`
		`void f(auto... vs);`

Homogeneous	`// OK:`
	`template <class Type, Type... vs>`
	`void f();`
		`// Ill-formed:`
		`template <class Type>`
		`void f(Type... vs);`

The absence of homogeneous function parameter packs is a source of confusion among programmers. A cursory search of Stack Overflow turned up several questions ([1], [2], [3], [4], [5], [6], [7], [8]) that basically amount to asking how to write a function with a homogeneous parameter pack. Some work-arounds are suggested:

Recursion

```
template <class T>
T min(T v)
{
    return v;
}

template <class T, class... Args>
T min(T v1, Args... vs)
{
    T v2 = min(vs...);
    return v2 < v1 ? v2 : v1;
}
```

std::initializer_list

```
template <class T>
T min(std::initializer_list<T> vs)
    [[expects: vs.size() > 0]]
{
    auto i = vs.begin();
    T v = *i++;
    for (; i != vs.end(); ++i)
    {
        if (*i < v)
            v = *i;
    }
```

	`    return v;` `}`

SFINAE	`template <class T, class... Args,`
	`          std::enable_if_t<(... && std::is_same_v<Args, T>),`
	`                           std::nullptr_t> = nullptr>`
	`T min(T v1, Args... vs)`
	`{`
	`    return (v1, ..., (v1 = vs < v1 ? vs : v1));`
	`}`

Concepts	`template <class T, class... Args>`
	`    requires (... && std::Same<Args, T>)`
	`T min(T v1, Args... vs)`
	`{`
	`    return (v1, ..., (v1 = vs < v1 ? vs : v1));`
	`}`

Homogeneous pack (this proposal)	`template <class T>`
	`T min(T v1, T... vs)`
	`{`
	`    return (v1, ..., (v1 = vs < v1 ? vs : v1));`
	`}`

The work-arounds suffer from a lack of clarity in the interface.

- The recursion approach advertises a function that can take variadic arguments of any type, but any attempt to call the function with a non-`T` will either cause the program to fail to compile, or lead to potentially unwanted implicit conversions.
- The `std::initializer_list` approach correctly advertises the accepted type, but is inflexible with respect to mutability and requires the user to enclose its arguments in braces. In the `min` example above, the contract precondition could be removed if an explicit argument preceded the `std::initializer_list`, but the syntactic separation remains, resulting in calls such as `min(a, { b, c, d })`.
- The SFINAE approach achieves the goal of constraining all arguments to be of the same type, but at the expense of readability, and the application of the `is_same_v` trait can disable desired conversions when constraining to a non-dependent parameter type.
- The Concepts approach is more readable than the SFINAE approach, but is otherwise interchangeable.

These patterns can be found in the Library Fundamentals TS in the form of `make_array`, and even in the standard for `std::min` and `std::max` and the deduction guide for `std::array`. All of these could arguably be written more naturally with homogeneous parameter packs.

Current	With homogeneous packs

`template <class T>`
`constexpr T min(initializer_list<T> t);`
 	
`template <class T, class Compare>`
`constexpr T min(initializer_list<T> t, Compare comp);`
	`template <class T>`
	`constexpr T min(T v1, T... vs);`
	
	`template <class Compare, class T>`
	`constexpr T min(Compare comp, T v1, T... vs);`

`template <class D = void, class... Types>`
`constexpr array<conditional_t<is_void_v<D>,`
`                                common_type_t<Types...>, D>,`
`                 sizeof...(Types)>`
`    make_array(Types&&... t);`
	`template <class T>`
	`constexpr auto make_array(T... t)`
	`    -> array<T, sizeof...(t)>;`

`// requires clause inferred from requirements`
`template<class T, class... U> requires (... && Same<T, U>)`
`array(T, U...) -> array<T, 1 + sizeof...(U)>;`
	`template<class T>`
	`array(T... t) -> array<T, sizeof...(t)>;`

Compared to `std::initializer_list`

This table shows some of the differences between homogeneous packs and `std::initializer_list`.

	`std::initializer_list`	homogeneous packs
Can be used outside of a template?	Yes	No
Can mutate elements?	No	Yes
Can appear before other arguments?	Yes	No
Get the number of elements	`x.size()`	`sizeof...(x)`
Iterate over the elements	Range-based `for`	Fold expressions

Homogeneous function parameter packs are *not* intended as a replacement for `std::initializer_list`. Although there are certainly areas of overlap ([9]), each has its place. Whereas a `std::initializer_list` forms a range over its component elements and is passed as a single argument, the elements of a parameter pack are passed as distinct arguments. This can render a constructor taking a parameter pack uninvocable if another constructor is deemed a better match. Consider the case of container initialization:

```
template <class T>
class MyContainer
{
public:
    MyContainer(std::initializer_list<T> elems);
```

```
    // ...
};
```

The `std::initializer_list` constructor allows an instance of `MyContainer` to be constructed using list-initialization from a list of element values:

```
MyContainer<int> cont = { 5, 10 };  // invokes MyContainer<int>::MyContainer(std::initializer_list<int>)
```

Under the rules governing uniform initialization, this syntax continues to work with homogeneous parameter packs:

```
template <class T>
class MyContainer
{
public:
    template <> MyContainer(const T&... elems);
    // ...
};

MyContainer<int> cont = { 5, 10 };  // invokes MyContainer<int>::MyContainer(const int&, const int&)
```

...*unless* there is a competing constructor that is a better match:

```
template <class T>
class MyContainer
{
public:
    template <> MyContainer(const T&... elems);
    explicit MyContainer(const T& min, const T& max);
    // ...
};

MyContainer<int> cont = { 5, 10 };  // error: MyContainer<int>::MyContainer(const int&, const int&) is explicit
```

With the additional constructor, it is now impossible to invoke the constructor containing the homogeneous parameter pack when there are exactly two elements, regardless of the initialization syntax chosen. In contrast, the constructor with `std::initializer_list` is *always* chosen when using list-initialization, and can be unambiguously selected (or not) when using direct non-list initialization.

Declaration	With `std::initializer_list`	With homogeneous packs

	`template <class T>`
	`class MyContainer`
	`{`
	`public:`
	`  MyContainer(std::initializer_list<T> elems);`
	`  explicit MyContainer(const T& min, const T& max);`
	`  // ...`
	`};`
		`template <class T>`
		`class MyContainer`
		`{`
		`public:`
		`  template <> MyContainer(const T&... elems);`
		`  explicit MyContainer(const T& min, const T& max);`
		`  // ...`
		`};`

`MyContainer<int> cont{1, 2, 3};`	Selects `std::initializer_list` constructor	Selects homogeneous pack constructor
`MyContainer<int> cont = { 1, 2, 3 };`	Selects `std::initializer_list` constructor	Selects homogeneous pack constructor
`MyContainer<int> cont(1, 2, 3);`	Error: no matching constructor	Selects homogeneous pack constructor
`MyContainer<int> cont({ 1, 2, 3 });`	Selects `std::initializer_list` constructor	Error: no matching constructor
`MyContainer<int> cont{1, 2};`	Selects `std::initializer_list` constructor	Selects `explicit` constructor
`MyContainer<int> cont = { 1, 2 };`	Selects `std::initializer_list` constructor	Error: selects `explicit` constructor
`MyContainer<int> cont(1, 2);`	Selects `explicit` constructor	Selects `explicit` constructor
`MyContainer<int> cont({ 1, 2 });`	Selects `std::initializer_list` constructor	Error: no matching constructor

For this reason, homogeneous packs are likely inappropriate for constructors. Homogeneous packs are likely the better option outside of constructors, where initializer lists must be separately enclosed in curly braces. One possible exception to this is when the API designer wishes to *move* or *forward* variadic constructor arguments rather than *copy* them. Frustratingly, the `std::initializer_list` template permits access to its elements only via references to `const`, whereas packs do not have this limitation.

Design Decisions

Declaring a homogeneous function parameter pack

A homogeneous function parameter pack is declared in exactly the same way as any other function parameter pack. The only difference is that the parameter declaration does not mention a template parameter pack:

```
template <class... T> void f(T... v);   // heterogeneous function parameter pack
template <class T>    void f(T... v);   // homogeneous function parameter pack
```

In fact, the pattern does not have to mention any template parameters at all:

```
void f(int... v);                       // homogeneous function parameter pack
```

Any function declaration that includes a homogeneous function parameter pack implicitly declares a function template.

The size of a homogeneous function parameter pack is deduced from function arguments at the call site. This requires the pack to be placed in a deduced context, which means that a function template can have at most one homogeneous function parameter pack, and the pack must appear at the end of the function parameter list. In all other respects, a homogeneous function parameter pack behaves no differently from any other parameter pack.

Lambdas

```
auto a = [](int... v) { return (1 * ... * v); };
```

The syntax for declaring a homogeneous function parameter pack in a lambda expression follows naturally from the syntax used for a function template. Because packs can only be declared in templates, adding a homogeneous pack declaration to a lambda expression will cause it to become a generic lambda.

Abbreviated function template syntax

EWGI Poll: This should be valid and declare a template: `void foo(int... v);`

SF	F	N	A	SA
4	6	4	0	1

With the recent adoption of P1141R2, function templates can now be declared without using the keyword `template`, provided that no template parameter names are needed within the declaration. These *abbreviated* function template declarations rely on placeholder types such as `auto` in the function signature to signal that they are templates.

Under this proposal, the presence of a homogeneous parameter pack in a function signature also indicates that a function declaration is an abbreviated function template declaration:

```
void foo(int... v);                     // OK, declares a template with an empty template parameter list
```

Under the proposed rules, any function declaration that includes a parameter pack is a template declaration. If the parameter pack is the only part of the function declaration requiring it to be a template and the pattern of the parameter pack does not contain any dependent names, then the template has no formal template parameters and the *template-head* must be omitted. To avoid confusion with explicit specializations, this proposal does not allow an empty *template-head* on a template declaration, even though the syntax is unambiguous in the presence of a function parameter pack declaration:

```
template <class T> void foo(T... v);        // #1 OK
void foo(int... v);                         // #2 OK, declares a template with an empty template parameter list
template <> void foo(float... v);           // Error: function parameter pack in explicit specialization
template <> void foo(int a);                // OK, declares an explicit specialization of #2
template <> void foo<>(int a, int b);       // OK, declares an explicit specialization of #2
template <> void foo<int>(int a, int b);    // OK, declares an explicit specialization of #1
template <> void foo(float a);              // OK, declares an explicit specialization of #1
```

Alternative: Allow empty template parameter lists in template declarations.

```
template <> void foo(float... v);           // OK, declares a template with a homogeneous parameter pack
```

Although this alternative does not introduce a true syntactic ambiguity with explicit specializations, the forms are similar enough that there is significant potential for programmer confusion. In an e-mail conversation with Richard Smith following the San Diego meeting, he indicated that he would be against the adoption of homogeneous function parameter packs under this model. EWGI had the same opinion in Kona:

EWGI Poll: This should be valid and declare a template: `template <> void foo(int... v);`

SF	F	N	A	SA
0	4	7	2	3

Homogeneous function parameter packs introduce a form of function template that has no formal template parameters. Without this option, these templates *cannot* be declared with the `template` keyword; the abbreviated syntax must be used instead. For consistency with other declarations, the idea of permitting the `template` keyword here is appealing; along with the next option, this could actually improve the overall consistency of the syntax surrounding template (and non-template) declarations.

Alternative: Modify explicit specialization syntax.

Explicit specialization declarations are inconsistent with the rest of the language. These declarations are prefixed with `template <>`, giving the appearance of a template declaration, but they don't actually declare templates. Explicit specializations are ordinary non-template entities that provide the definition for a particular specialization of a template, and the template specialization must be named in (or deducible from) the declaration. Explicit and partial specializations are the only declarations where a *template-id* may appear as the name of a declared entity. This makes the `template <>` introducer both misleading and redundant.

The `template <>` introducer can be dropped without any reduction in expressivitiy:

```
template <class T> class C; // class template
template <> class C<int>;   // ok, explicit specialization (proposed: not deprecated)
class C<int>;               // proposed: explicit specialization
```

```
template <int V> constexpr int v = V;   // variable template
template <> constexpr int v<1> = 0;     // ok, explicit specialization (proposed: deprecated)
constexpr int v<1> = 0;                 // proposed: explicit specialization

template <class T, class U> void f(T t, U u);       // function template
template <> void f<int, double>(int i, double d);   // ok: explicit specialization (proposed: deprecated)
template <> void f<>(int i, double d);              // ok: explicit specialization (proposed: deprecated)
template <> void f(int i, double d);                // ok: explicit specialization (proposed: deprecated)
void f<int, double>(int i, double d);               // proposed: explicit specialization
void f<>(int i, double d);                          // proposed: explicit specialization
void f(int i, double d);                            // ok: ordinary function declaration (proposed: no change)
```

Under this approach, the misleading part of the syntax is removed, and the result is a more consistent language. Non-template entities no longer look like templates. For explicit specializations of function and variable templates, the old syntax can be deprecated. (The old syntax for explicit specializations of class templates probably cannot be deprecated at this time, as their use is *very* common.) Anecdotally, explicit specializations of function templates are rare and their use is generally discouraged due to subtleties around overloading.

EWGI in Kona opted not to poll this option, but the general sentiment of the room was that a separate paper exploring this idea would be welcome.

Alternative: Require a sigil in the template parameter list to indicate that a homogeneous pack will be present in the function declaration.

```
template <...> void foo(int... v);
template <class T, ...> void bar(T... v);
```

This idea, including the use of the ellipsis as the sigil, was suggested by Richard Smith in an impromptu conversation during the San Diego meeting. He and David Vandevoorde both indicated that this requirement would make the feature simpler to implement across a broad range of compilers. This notion is not directly supported by the experience of implementing the feature in Clang, but there is as yet no implementation experience with any other compiler. In Clang, the presence of an unparenthesized ellipsis in a declarator causes the compiler to treat the declaration as having a particular dependent type corresponding to a parameter pack, and the presence of a parameter of that type is enough to differentiate a function declaration as a template declaration instead of an explicit specialization or ordinary function declaration.

P1141R2, adopted in San Diego, adds support for function template declarations that lack a template head entirely; in this case, the presence of the `auto` keyword in the function parameter list indicates that the function declaration declares a function template. Any implementation that can handle these new declarations should also be able to handle homogeneous packs without added syntax. In a follow-up conversation with Richard over e-mail, he agreed with this conclusion.

Alternative: Allow the angle brackets to be omitted when no template parameters are needed.

```
template void foo(int... v);
```

This removes the apparent conflict with explicit specialization syntax, but introduces the same apparent conflict with the syntax for explicit instantiations. In both cases, there is no actual conflict; the compiler can tell that the function is a template by the presence of the parameter pack. This approach was considered and rejected because it introduces an irregularity into the grammar. Currently, all templates introduced with the `template` keyword must include a parameter list enclosed in angle brackets; removing them would be akin to removing the parentheses on a parameterless function declaration. The only declaration that uses the `template` keyword without angle brackets is the explicit instantiation declaration, and those do not declare templates.

Alternative: Allow or require a template parameter for the size of the pack.

```
template <size_t Len> void foo(int...[Len] v);
```

This would allow the size of the homogeneous pack to be explicitly specified, which could be useful in some situations. Making the length of the pack a formal template parameter would also make clear in the syntax that the length is an axis of specialization; under the currently proposed rules, this is still true, but the fact is hidden from the user, and the length can only be deduced rather than specified.

The drawback to this approach is a matter of regularity. While the language allows the user to query the size of a pack using the `sizeof...` operator, it does not currently allow the size to be explicitly specified. If this facility were added, it could reasonably be applied to homogeneous template parameter packs as well as homogeneous function parameter packs. It would make little sense to try to apply this to heterogeneous packs, since the size must match the number of arguments passed to the template parameter pack. Since this facility would necessarily be optional for homogeneous template parameter packs, consistency demands that it should also be optional for homogeneous function parameter packs.

Alternative: Allow empty angle brackets even when there is no homogeneous parameter pack.

```
template <> void foo(int n);
```

Under the first alternative to the proposed rules, the *template-parameter-list* is optional in a *template-head* in order to permit homogeneous packs with non-dependent types. The alternative rules also add semantic constraints requiring at least one template parameter whenever there is no homogeneous pack. The semantic constraints could be relaxed, allowing trivial templates that only permit vacuous specialization. This would not be very useful; these trivial templates would behave like normal non-template entities in almost every way imaginable, aside from syntactic minutiae (for instance, a trivial function template could be defined in a header file without using the `inline` keyword).

This alternative also gives rise to a genuine ambiguity with explicit specializations:

```
template <> void f(int v);      // #1: trivial function template
template <> void f(int... vs);  // #2: function template with homogeneous pack
template <> void f(int v);      // #3: redeclaration of #1 or explicit specialization of #2?
```

Apart from minor simplifications in the language specification, about the only "benefit" of this approach would be to make the token sequence `[]<>(){}` well-formed for the amusement of language nerds.

The Oxford variadic comma

EWGI Poll: Make vararg declaration comma mandatory.

SF	F	N	A	SA
10	6	1	0	0

In the C programming language, the appearance of an ellipsis in the *parameter-type-list* of a function declarator indicates that the function accepts a variable number of arguments of varying types following the last formal parameter in the list. Such an ellipsis will henceforth be referred to as a *varargs ellipsis* to distinguish it from the ellipsis used in the declaration of a parameter pack. To be syntactically valid in C, a varargs ellipsis must be preceded by at least one parameter declaration and an intervening comma:

> *parameter-type-list*:
> *parameter-list*
> *parameter-list* `,` `...`

C++ has the same behavior, but with an expanded syntax, requiring neither the preceding parameter declaration nor the intervening comma:

> *parameter-declaration-clause*:
> *parameter-declaration-list*$_{opt}$ `...`$_{opt}$
> *parameter-declaration-list* `,` `...`

The varargs ellipsis was originally introduced in C++ along with function prototypes. At that time, the feature did not permit a comma prior to the ellipsis. When C later adopted these features, the syntax was altered to require the intervening comma, emphasizing the distinction between the last formal parameter and the varargs parameters. To retain compatibility with C, the C++ syntax was modified to permit the user to add the intervening comma. Users therefore can choose to provide the comma or leave it out.

When paired with function parameter packs, this creates a syntactic ambiguity that is currently resolved via a disambiguation rule: When an ellipsis that appears in a function parameter list might be part of an abstract (nameless) declarator, it is treated as a pack declaration if the parameter's type names an unexpanded parameter pack or contains `auto`; otherwise, it is a varargs ellipsis. At present, this rule effectively disambiguates in favor of a parameter pack whenever doing so produces a well-formed result.

Example (status quo):

```
template <class... T>
void f(T...); // declares a variadic function template with a function parameter pack

template <class T>
void f(T...); // same as void f(T, ...)
```

With homogeneous function parameter packs, this disambiguation rule needs to be revisited. It would be very natural to interpret the second declaration above as a function template with a homogeneous function parameter pack, and that is the resolution proposed here. By requiring a comma between a parameter list and a varargs ellipsis, the disambiguation rule can be dropped entirely, simplifying the language without losing any functionality or degrading compatibility with C.

This is a breaking change, but likely not a very impactful one. In the parlance of P0684R2, this would be a "Very Good Change": Compilers can issue warnings in C++20 mode (or earlier) whenever the disambiguation rule is resolved in favor of a varargs ellipsis, and the warning can be resolved without any change in meaning by the addition of a single character. Moreover, the number of instances should be vanishingly small. In modern C++ code, the varargs ellipsis has largely been superseded by function parameter packs. Today, apart from SFINAE uses where the disambiguation rule doesn't apply, the varargs ellipsis can mainly be found in header files that are intended to be consumed by both C and C++ code, in order to facilitate interoperability between the two languages. In these cases, the declarations must conform to the rules imposed by C syntax, and so they will already conform to the rules proposed here. Lastly, personal experience suggests that the vast majority of C++ users aren't even aware that the comma preceding a varargs ellipsis is optional.

The impact of a breaking change

The ACTCD19 dataset at codesearch.isocpp.org ([10]) contains a vast amount of source code collected from various open-source projects. According to the codesearch FAQ, the dataset comprises 876,800,403 lines of source code across 2,489,599 files. This dataset was used to estimate the impact of requiring a comma before a varargs ellipsis.

Identifying uses of varargs ellipses is decidedly non-trivial. Since this is an area of syntactic ambiguity, only a C++ parser with sufficient knowledge of the language can identify all occurrences and distinguish them from parameter pack declarations. Such a parser would also require special build scripts for each of the projects represented in the dataset. In the absence of those resources, a mix of regular expression searches and human review was used instead.

Only header files (those with extensions of `.h`, `.hh`, `.hpp`, or `.hxx`) were considered. In order to discard comments and macro definitions, each source file was stripped of preprocessor directives and then run through the C++ preprocessor, reducing the search space to syntactically significant text. (This process still left behind a significant number of macro invocations, because the macro definitions were no longer visible after stripping preprocessor directives. Inactive code blocks, such as those guarded by `#if 0`, were also retained.) The collective output was then scanned for lines containing ellipses, resulting in 258,196 lines of interesting text, which were then tagged according to the file each line originated in.

A variety of patterns were identified based on common identifiers, operators, and other textual hints that typically declare variadic packs or expand them; these were excluded from further analysis.

From these results, 89,614 lines contain a comma immediately preceding an ellipsis; those lines unambiguously declare varargs functions.

8,101 lines begin with an ellipsis, and further analysis showed that 6,241 of those lines appeared to come from function declarations. Of those, 6,092 are immediately preceded by a comma on a prior line; from the remaining 149 lines, 131 are catch clauses or function declarations containing only an ellipsis. The last 18 lines have macros mixed in with the parameter list for static analysis purposes. Every single one of these ellipses is unambiguous with respect to pack declarations.

From the entire result set, 6,318 lines were positively identified as containing a varargs ellipsis *without* a preceding comma.

Lines containing ellipses	258,196
Varargs ellipses, with comma	95,724
Varargs ellipses, without comma	6,318
From boost	6,182
From boost (deduplicated)	879
All others	136

Of the 6,318 confirmed varargs declarations without a preceding comma, 6,182 come from a single source: the boost library. Boost is a very popular general-purpose lirary in the C++ ecosystem, and several of the open-source projects in the dataset contain their own independent copies. In total, the dataset contains seventeen separate complete or partial copies of various versions of boost. After deduplication, the 6,182 results found in boost are reduced to 879.

The 879 unique hits in boost tend to follow a generic pattern; nearly half (416 hits) come from `type_traits/detail/is_mem_fun_ptr_tester.hpp`. This one file defines a family of function templates (all named `is_mem_fun_pointer_tester`), each taking a function pointer of progressively higher arity, from 0 to 24, in every possible combination of qualifiers and calling convention, all both with and without a varargs ellipsis. The intent is to allow generic code to detect at compile time whether or not a given type is a member function pointer type. In modern C++, using variadic templates with class template partial specialization, this 2,759 line file can be reduced to just a small handful of lines. Most of the other hits in boost follow a similar pattern, and are intended to facilitate generic programming in pre-C++11 modes.

Boost is very actively maintained and aggressively pursues compatibility with a wide range of compilers and standards. If the comma preceding a varargs ellipsis becomes mandatory, boost will adopt the change very quickly.

Wording

All modifications are presented relative to N4830. "*[...]*" indicates elided content that is to remain unchanged.

Modify §3.21 [defns.signature.spec]:

> **signature**
> ⟨function template specialization⟩ signature of the template of which it is a specialization ~~and~~, its template arguments (whether explicitly specified or deduced), and the size of its trailing homogeneous function parameter pack ([temp.variadic]) (if any)

Modify §3.24 [defns.signature.member.spec]:

> **signature**
> ⟨class member function template specialization⟩ signature of the member function template of which it is a specialization ~~and~~, its template arguments (whether explicitly specified or deduced), and the size of its trailing homogeneous function parameter pack ([temp.variadic]) (if any)

Modify §7.5.5 [expr.prim.lambda] paragraph 5:

> A lambda is a *generic lambda* if ~~there is a *decl-specifier* that is a *placeholder-type-specifier* in the *decl-specifier-seq* of a *parameter-declaration* of the *lambda-expression*, or if the lambda has a *template-parameter-list*.~~
>
> — there is a *decl-specifier* that is a *placeholder-type-specifier* in the *decl-specifier-seq* of a *parameter-declaration* of the *lambda-expression*,
>
> — the lambda has a *template-parameter-list*, or
>
> — the lambda has a *lambda-declarator* and any *parameter-declaration* declares a parameter pack ([temp.variadic]).
>
> [*Example:*
>
> ```
> int i = [](int i, auto a) { return i; }(3, 4);          // OK: a generic lambda
> int j = []<class T>(T t, int i) { return i; }(3, 4);    // OK: a generic lambda
> int k = [](int... i) { return (0 + ... + i); }(3, 4);   // OK: a generic lambda
> ```
>
> — *end example*]

Modify §9.2.3.5 [dcl.fct] paragraph 3:

> *[...]*
>
> *parameter-declaration-clause*:
> *parameter-declaration-list*$_{opt}$ ~~...~~ ~~$_{opt}$~~

`...`
parameter-declaration-list `,` `...`

[...]

Modify §9.2.3.5 [dcl.fct] paragraph 4:

[...] If the *parameter-declaration-clause* terminates with an ellipsis or a function parameter pack ([temp.variadic]), the number of arguments shall be equal to or greater than the number of parameters that do not have a default argument and are not function parameter packs. ~~Where syntactically correct and where "`...`" is not part of an *abstract-declarator*, "`, ...`" is synonymous with "`...`".~~ [*Example:* The declaration

```
int printf(const char*, ...);
```

declares a function that can be called with varying numbers and types of arguments.

```
printf("hello world");
printf("a=%d b=%d", a, b);
```

However, the first argument must be of a type that can be converted to a const char* — *end example*] [*Note:* The standard header `<cstdarg>` contains a mechanism for accessing arguments passed using the ellipsis (see [expr.call] and [support.runtime]). — *end note*]

Modify §9.2.3.5 [dcl.fct] paragraph 18:

An *abbreviated function template* is a function declaration whose *parameter-type-list* includes one or more placeholders ([dcl.spec.auto]) or a trailing homogeneous function parameter pack ([temp.variadic]). *[...]*

Modify §9.2.3.5 [dcl.fct] paragraph 21:

A *declarator-id* or *abstract-declarator* containing an ellipsis shall only be used in a *parameter-declaration*. When it is part of a *parameter-declaration-clause*, the *parameter-declaration* declares a function parameter pack ([temp.variadic]). Otherwise, the *parameter-declaration* is part of a *template-parameter-list* and declares a template parameter pack; see [temp.param]. A function parameter pack is a pack expansion ([temp.variadic]) when the type of the parameter contains one or more template parameter packs that have not otherwise been expanded. [*Example:*

```
template<typename... T> void f(T (* ...t)(int, int));

int add(int, int);
float subtract(int, int);

void g() {
  f(add, subtract);
}
```

— *end example*]

Delete §9.2.3.5 [dcl.fct] paragraph 22 and the accompanying footnote:

~~There is a syntactic ambiguity when an ellipsis occurs at the end of a *parameter-declaration-clause* without a preceding comma. In this case, the ellipsis is parsed as part of the *abstract-declarator* if the type of the parameter either names a template parameter pack that has not been expanded or contains `auto`; otherwise, it is parsed as part of the *parameter-declaration-clause*.~~

Modify §12.4 [over.over] paragraph 2:

If the name is a function template, template argument deduction is done ([temp.deduct.funcaddr]), and if the argument deduction succeeds, ~~the resulting template argument list is used to generate~~ a single function template specialization~~, which~~ is generated using the resulting template argument list and the deduced number of elements for the trailing homogeneous function parameter pack (if present). The generated function template specialization is then added to the set of overloaded functions considered. [*Note:* As described in [temp.arg.explicit], if deduction fails and the function template name is followed by an explicit template argument list, the *template-id* is then examined to see whether it identifies a single function template specialization. If it does, the *template-id* is considered to be an lvalue for that function template specialization. The target type is not used in that determination. — *end note*]

Modify §13.3 [temp.arg] paragraph 4:

When a template declares no *template-parameter*s, or when template argument packs or default *template-argument*s are used, a *template-argument* list can be empty. In that case the empty <> brackets shall still be used as the *template-argument-list*. [*Example:*

```
template<class T = char> class String;
String<>* p;                    // OK: String<char>
String* q;                      // syntax error
template<class ... Elements> class Tuple;
```

```
  Tuple<>* t;                    // OK: Elements is empty
  Tuple* u;                      // syntax error
```
— *end example*]

Delete §13.6 [temp.decls] paragraph 3 (redundant, moved to [temp.alias]):

~~Because an *alias-declaration* cannot declare a *template-id*, it is not possible to partially or explicitly specialize an alias template.~~

Modify §13.6.3 [temp.variadic] paragraph 5:

A pack expansion consists of a pattern and an ellipsis, the instantiation of which produces zero or more instantiations of the pattern in a list (described below). The form of the pattern depends on the context in which the expansion occurs. Pack expansions can occur in the following contexts:

— In a function parameter pack that is a pack expansion ([dcl.fct]); the pattern is the *parameter-declaration* without the ellipsis.

[...]

Insert a new paragraph after §13.6.3 [temp.variadic] paragraph 5:

A function parameter pack whose declaration is not a pack expansion is a *homogeneous function parameter pack*. A homogeneous function parameter pack shall only appear at the end of the *parameter-declaration-clause* of a function template, member function template, or generic lambda. [*Note:* Because a homogeneous function parameter pack has no corresponding template parameter packs, the length of the pack must be deduced at the point of use; therefore, the pack cannot appear in a non-deduced context. — *end note*]

Modify §13.6.3 [temp.variadic] paragraph 10:

[...] [*Example:*

```
~~template<typename ...Args>~~
bool all(~~Args~~bool ...args) { return (... && args); }
bool b = all(true, true, true, false);
```

Within the instantiation of `all`, the returned expression expands to `((true && true) && true) && false`, which evaluates to `false`. — *end example*] *[...]*

Modify §13.6.7 [temp.alias] paragraph 1:

A *template-declaration* in which the declaration is an *alias-declaration* ([dcl.dcl]) declares the identifier to be an *alias template*. An alias template is a name for a family of types. The name of the alias template is a *template-name*. [*Note:* Because an *alias-declaration* cannot declare a *template-id*, it is not possible to partially or explicitly specialize an alias template. — *end note*]

Modify §13.7.2 [temp.dep] paragraph 1:

[...] An expression may be *type-dependent* (that is, its type may depend on a template parameter or the number of elements in a parameter pack) or *value-dependent* (that is, its value when evaluated as a constant expression ([expr.const]) may depend on a template parameter or the number of elements in a parameter pack) as described in this subclause. *[...]*

Modify §13.7.2.1 [temp.dep.type] paragraph 9:

A type is dependent if it is

— a template parameter,

— a member of an unknown specialization,

— a nested class or enumeration that is a dependent member of the current instantiation,

— a cv-qualified type where the cv-unqualified type is dependent,

— a compound type constructed from any dependent type,

— an array type whose element type is dependent or whose bound (if any) is value-dependent,

— a function type whose parameter-type-list contains a parameter pack,

— a function type whose exception specification is value-dependent,

— denoted by a *simple-template-id* in which either the template name is a template parameter or any of the template arguments is a dependent type or an expression that is type-dependent or value-dependent or is a pack expansion [*Note:* This includes an *injected-class-name* ([class]) of a class template used without a *template-argument-list*. — *end note*] , or

— denoted by `decltype(`*expression*`)`, where *expression* is type-dependent ([temp.dep.expr]).

Modify §13.9.2 [temp.deduct] paragraph 1:

When a function template specialization is referenced, all of the template arguments shall have values and the number of elements in each function parameter pack shall be known. The ~~values~~template arguments can be explicitly specified or, in some cases, be deduced from the use or obtained from default *template-argument*s. The number of elements in a homogeneous function parameter pack must be deduced. [*Note:* If a function template declaration includes a homogeneous function parameter pack, then a *template-id* is never sufficient to refer to an individual specialization of the template. — *end note*] [*Example:*

```
template<class T> class Array { /* ... */ };
template<class T> void sort(Array<T>& v);

void f(Array<dcomplex>& cv, Array<int>& ci) {
  sort(cv);                   // calls sort(Array<dcomplex>&)
  sort(ci);                   // calls sort(Array<int>&)
}
```

~~and~~

```
template<class U, class V> U convert(V v);

void g(double d) {
  int i = convert<int>(d);    // calls convert<int,double>(double)
  int c = convert<char>(d);   // calls convert<char,double>(double)
}

int sum(int... n) { return (0 + ... + n); }

void h(int x, int y, int z) {
  int i = sum(x, y, z);       // calls sum<>(int,int,int)
}
```

— *end example*]

Modify §13.9.2 [temp.deduct] paragraph 5:

[...] When all template arguments have been deduced or obtained from default template arguments, all uses of template parameters in the template parameter list of the template and the function type are replaced with the corresponding deduced or default argument values. If the function type has a homogeneous function parameter pack and the number of elements for the pack has been deduced, the pack is replaced with a corresponding number of function parameters, each an instance of the pack's pattern. If the substitution results in an invalid type, as described above, type deduction fails. If the function template has associated constraints ([temp.constr.decl]), those constraints are checked for satisfaction ([temp.constr.constr]). If the constraints are not satisfied, type deduction fails.

Modify §13.9.2 [temp.deduct] paragraph 11:

[*Note:* Type deduction may fail for the following reasons: *[...]*

— Attempting to deduce the type of a function template specialization from a *template-id* when the function template contains a homogeneous function parameter pack. [*Example:*

```
template <class T> void f(T... v);
auto p = &f<int>;   // ambiguous; function parameter pack of unknown size
```

— *end example*]

— *end note*]

Modify §13.9.2.1 [temp.deduct.call] paragraph 1:

[...] For a function parameter pack that occurs at the end of the *parameter-declaration-list*, the number of elements in the pack is determined as the number of arguments remaining in the call. ~~d~~Deduction is performed for each remaining argument ~~of the call~~, taking the type `P` of the *declarator-id* of the function parameter pack as the corresponding function template parameter type. Each deduction deduces template arguments for subsequent positions in the template parameter packs expanded by the function parameter pack (if any). When a function parameter pack appears in a non-deduced context ([temp.deduct.type]), the type of that pack is never deduced. [*Example:*

```
template<class ... Types> void f(Types& ...);
template<class T1, class ... Types> void g(T1, Types ...);
template<class T1, class ... Types> void g1(Types ..., T1);
void h(int x, float& y) {
  const int z = x;
  f(x, y, z);                  // Types is deduced to int, float, const int
  g(x, y, z);                  // T1 is deduced to int; Types is deduced to float, int
  g1(x, y, z);                 // error: Types is not deduced
  g1<int, int, int>(x, y, z);  // OK, no deduction occurs
}
```

— end example]

Modify §13.9.2.4 [temp.deduct.partial] paragraph 8:

> Using the resulting types `P` and `A`, the deduction is then done as described in [temp.deduct.type]. If `P` is a function parameter pack, the type `A` of each remaining parameter type of the argument template is compared with the type `P` of the *declarator-id* of the function parameter pack. Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by the function parameter pack (if any). *[...]*

Modify §13.9.2.5 [temp.deduct.type] paragraph 10:

> *[...]* If the *parameter-declaration* corresponding to P_i is a function parameter pack, then the type of its *declarator-id* is compared with each remaining parameter type in the *parameter-type-list* of `A`. Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by the function parameter pack (if any). *[...]*

Modify the definition of the `__cpp_variadic_templates` macro in §15.10 [cpp.predefined] table 17:

Table 17 — Feature-test macros

Macro name	Value
[...]	
`__cpp_variadic_templates`	~~`200704L`~~`201911L`
[...]	

Acknowledgments

Thanks to Richard Smith, David Vandevoorde, and John Spicer for providing some valuable impromptu feedback. Thanks also to Zhihao Yuan for pointing out an issue with the `make_arary` example.

References

1. Specifying one type for all arguments passed to variadic function or variadic template function w/out using array, vector, structs, etc? (https://stackoverflow.com/questions/3703658/)
2. C++ parameter pack, constrained to have instances of a single type? (https://stackoverflow.com/questions/38528801/)
3. Variadic template parameters of one specific type (https://stackoverflow.com/questions/30773216/)
4. C++ parameter pack with single type enforced in arguments (https://stackoverflow.com/questions/47470874/)
5. C++11 variable number of arguments, same specific type (https://stackoverflow.com/questions/18017543/)
6. Enforce variadic template of certain type (https://stackoverflow.com/questions/30346652/)
7. variadic template of a specific type (https://stackoverflow.com/questions/13636290/)
8. C++11: Variadic Homogeneous Non-POD Template Function Arguments? (https://stackoverflow.com/questions/9762531/)
9. Bjarne Stroustrup, Variadic Templates (https://parasol.tamu.edu/people/bs/622-GP/variadic-templates-and-tuples.pdf)
10. Andrew Tomazos, codesearch.isocpp.org FAQ (https://codesearch.isocpp.org/faq.html)